

项目开发路线图

📌 我这里补充一下整个项目的架构、目录，开发路线。这样想敲代码的同学就可以上手开发了，不然可能会没头绪。同时，这也方便大家更好地理解项目。

1. 技术栈

组件	技术	版本	用途
运行时	Java	21	具有增强性能的现代 J
框架	Spring Boot	3.2.4	核心应用框架
数据库	MySQL	8	主数据存储
缓存	Redis	7	缓存和会话管理
安全	Spring Security	6.x	认证和授权
消息队列	Apache Kafka	Latest	事件驱动架构
对象存储	Aliyun OSS	3.17.3	文件和媒体存储
AI 集成	Spring AI	1.0.0-SNAPSHOT	LLM 服务集成
构建工具	Maven	Latest	依赖管理

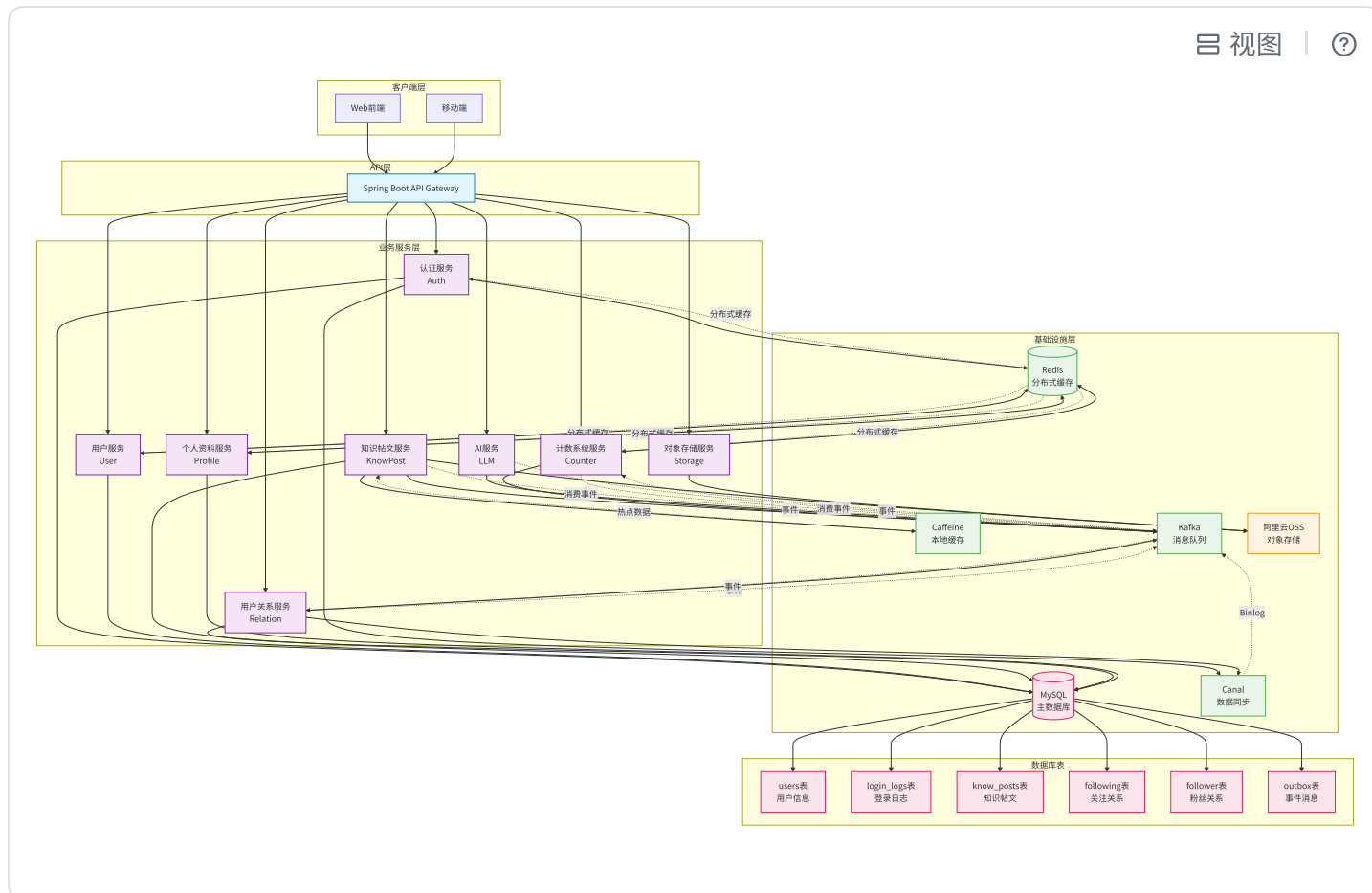
2. 项目目录

代码块

```
1  zhiguang/
2  |— src/main/java/com/tongji/
3  |   |— ZhiGuangApplication.java # 项目启动文件
4  |   |— auth/                    # 认证，登录注册
5  |   |— profile/                 # 用户资料管理
6  |   |— knowpost/               # 知识内容系统
7  |   |— counter/                # 计数与分析
8  |   |— relation/              # 用户关系
9  |   |— storage/                # 对象存储服务
10 |   |— llm/                    # AI/LLM 集成
11 |   |— cache/                  # 缓存基础设施
12 |   |— config/                 # 配置管理
13 |— db/
```

```
14 | schema.sql # 数据库模式
15 | pom.xml # Maven 配置
16
```

3. 项目架构



核心特点:

- 分层架构: 标准的Controller-Service-Repository三层架构
- 事件驱动: 使用Kafka实现异步处理和解耦
- 多级缓存: Caffeine本地缓存 + Redis分布式缓存
- 数据同步: Canal监听MySQL binlog同步到Kafka
- AI集成: Spring AI + Deepseek模型提供内容生成能力

4. 开发路线

第一阶段: 基础环境搭建

1. 项目初始化

- 创建Spring Boot 3.2.4项目, 配置 [pom.xml] 基础依赖

- 配置数据库连接，创建 [db/schema.sql] 表结构
- 配置 Redis 连接和基础缓存

2. 核心配置

- [安全配置] (auth/config/**SecurityConfig.java**) - Spring Security基础配置
- [线程池配置] (config/**ThreadPoolConfig.java**) - 异步任务处理
- [缓存配置] (cache/config/**CacheConfig.java**) - Caffeine和Redis配置

第二阶段：用户认证系统

3. 用户基础功能

- [用户实体和数据库操作] (user/domain/**User.java**) - 用户模型
- [用户服务] (user/service/**UserService.java**) - 注册、查询、更新

4. 认证授权

- [JWT Token管理] (auth/token/**JwtService.java**) - 令牌生成和验证
- [认证服务] (auth/service/**AuthService.java**) - 登录逻辑
- [验证码系统] (auth/verification/**VerificationService.java**) - 短信/邮箱验证

第三阶段：核心业务功能

5. 个人资料管理

- [个人资料服务] (profile/service/**ProfileService.java**) - 用户信息展示和编辑
- [API接口] (profile/api/**ProfileController.java**) - RESTful API设计

6. 知识帖文系统

- [帖文模型] (knowpost/model/**KnowPost.java**) - 内容结构设计
- [ID生成器] (knowpost/id/**SnowflakeIdGenerator.java**) - 分布式ID生成
- [帖文服务] (knowpost/service/**KnowPostService.java**) - CRUD

- [对象存储集成] (storage/**OssStorageService.java**) - 文件上传

第四阶段：社交与互动

7. 用户关系管理

- [关系服务] (relation/service/**RelationService.java**) - 关注/粉丝逻辑
- [事件处理] (relation/event/**RelationEvent.java**) - 关系变更事件

8. 计数系统

- [计数器设计] (counter/schema/**CounterSchema.java**) - 多维度计数
- [事件驱动计数] (counter/event/**CounterEvent.java**) - 异步计数更新
- [热键检测] (cache/hotkey/**HotKeyDetector.java**) - 热点数据识别

第五阶段：高级特性

9. 消息队列集成

- [Kafka事件生产者] (counter/event/**CounterEventProducer.java**) - 事件发送
- [事件消费者] (relation/outbox/**CanalOutboxConsumer.java**) - 事件处理
- [Canal数据同步] (relation/outbox/**CanalKafkaBridge.java**) - 数据库变更同步

10. AI功能集成

- [LLM服务配置] (llm/**LlmConfig.java**) - Deepseek模型集成
- [内容生成服务] (llm/service/**KnowPostDescriptionService.java**) - AI辅助内容创作

第六阶段：性能优化

11. 缓存策略

- [Feed缓存服务] (knowpost/service/**FeedCacheService.java**) - 动态流缓存
- [多级缓存实现] (cache/config/**CacheConfig.java**) - 本地+分布式缓存

12. 动态流架构

- [Feed服务] (knowpost/service/**KnowPostFeedService.java**) - 推荐算法基础
- [缓存失效监听] (knowpost/listener/**FeedCacheInvalidationListener.java**) - 缓存一致性

学习重点建议

技术深度递进：

1. 从 [Spring Boot基础] (3-environment-setup)开始，理解IoC和AOP
2. 深入 [Spring Security] (5-jwt-token-management)的安全机制
3. 掌握 [事件驱动架构] (13-event-driven-architecture)设计模式
4. 理解 [分布式缓存策略] (15-cache-strategy-implementation)